

Lab 1

Name: Abdulrahim Kuteifan

Milkias Michael Teklesenbet

Questions

5) In NDC, the x, y, z coordinates have a limit of $[-1, 1]$. Exceeding that will place the object outside the viewing frustum, thus clipping the object and would not be rendered. This is because the graphics pipeline uses NDC to determine what is visible, and any part of an object that falls outside this standard cube is considered off-screen and not rendered.

6) Without a vbo, we would have to duplicate the vertices, so each triangle lists its three vertices explicitly. We would get 36 vertices for the 12 triangles that make up the cube. This uses more memory and duplicates vertices data, which is not so efficient.

The model, projection, view matrices are printed out as follows:

```
mat4x4((1.810660, 0.000000, 0.000000, 0.000000), (0.000000, 2.414213, 0.000000, 0.000000),  
(0.000000, 0.000000, -1.020202, -1.000000), (0.000000, 0.000000, -2.020202, 0.000000))
```

7) The model matrix in this lab is the multiplication of the two rotation matrices (rotation around the x-axis and y-axis) and the matrices around the x-axis and y-axis looks as shown below:

```
GLfloat rotate_y[4][4] = {  
    cy,    0.0,    sy,    0.0,  
    0.0,    1.0,    0.0,    0.0,  
    -sy,    0.0,    cy,    0.0,  
    0.0,    0.0,    0.0,    1.0  
};  
GLfloat rotate_x[4][4] = {  
    1.0,    0.0,    0.0,    0.0,  
    0.0,    cx,    -sx,    0.0,  
    0.0,    sx,     cx,    0.0,  
    0.0,    0.0,    0.0,    1.0  
};
```

The sine and cosine part in the matrix that rotates the cube around y-axis simply rotates the cube around the y-axis. The middle row $[0.0 \ 1.0 \ 0.0 \ 0.0]$ keeps the y unchanged since the rotation is around the y-axis. The last column is simply $[0.0 \ 0.0 \ 0.0 \ 1.0]$ and that means we have no translation i.e. the coordinates of the cube are unchanged (the cube is kept centered at the origin in our case).

Using the same explanation as with the rotation around the y-axis, the matrix that handles the rotation around the x-axis uses the sine and cosine to rotate the cube around the x-axis and the last column is still $[0.0 \ 0.0 \ 0.0 \ 1.0]$ as with the rotation around the y-axis, which means the cube's coordinates are unchanged.

Since there are no factors that multiplies the sine and cosine parts of the matrices, we are simply not scaling the edges of the cube.

So, the model matrix simply rotates the cube around the x-axis and y-axis and do not scale or translate it. This model matrix is used to map the cube from its local/object space to the world space.

8) The view matrix in this lab looks like as shown below:

```
GLfloat inverseViewMatrix[4][4] = {  
    1.0, 0.0, 0.0, 0.0,  
    0.0, 1.0, 0.0, 0.0,  
    0.0, 0.0, 1.0, -2.0,  
    0.0, 0.0, 0.0, 1.0  
};
```

The upper left 3x3 part of the view matrix is an identity matrix which means there is neither rotation nor scaling. The last column of the matrix is [0.0 0.0 -2.0 1.0] i.e. there is a translation along the z-axis and that translation moves the whole world points by -2 units in the z-axis when going to the view space.

This matrix is used to move from the world space to the view space.

9) The projection matrix is defines as shown below:

```
GLfloat projectionMatrix[4][4] = {  
    h_over_w, 0.0, 0.0, 0.0,  
    0.0, 1.0, 0.0, 0.0,  
    0.0, 0.0, A, B,  
    0.0, 0.0, -1.0, 0.0  
};
```

The top left of the matrix (h_over_w) is used to compensate for the window's aspect ratio so that the cube doesn't look stretched when the window is resized.

A and B in the third-row control how z (depth) is mapped from view space to clip/NDC space. They are chosen so that points near plane (z = -1 in this lab) get mapped to z_ndc = -1

and points at the far plane ($z = -100$ in this lab) get mapped to $z_{ndc} = +1$. They don't translate or rotate in the usual geometric sense, but they remap depth.

The -1 at the bottom row is used to add perspective to the cube, i.e. the part of the cube that lies farther away will look smaller compared to the part that lies closer.

10) Other `glDepthFunc` options are: `GL_LEQUAL`, `GL_GREATER`, `GL_GEQUAL`, `GL_EQUAL`, `GL_NOTEQUAL`, `GL_ALWAYS`, `GL_NEVER`. The one we used (`GL_LESS`) is the right choice with the standard depth buffer where it keeps the nearest fragments and avoids depth ties overwriting each other. Picking a different test can invert depth (`GL_GREATER/GL_GEQUAL`), ignore depth entirely (`GL_ALWAYS`), or over-reject due to precision (`GL_EQUAL/GL_NEVER`). This can cause far geometry to cover near ones, cause z-fighting, or missing surfaces.